

CTP FINAL PROJECT REPORT

Project Title: Detective Case Solver

Student 1 Details

Name: Suvodra Chakma

ID: 242161

AUW Email: auw242161@auw.edu.bd

Student 2 Details

Name: Mittra Chakma

ID: 242313

AUW Email: auw242313@auw.edu.bd

Student 3 Details

Name: F.Lallawmzuali

ID: 242169

AUW Email: auw242169@auw.edu.bd

1. Abstract

This project presents an interactive web-based detective simulation game titled "**Detective Case Solver.**" Developed using HTML5, CSS3, and modern JavaScript, the game challenges players to solve a mystery involving three primary suspects—Mittra, Lomi, and Suvodra by examining evidence on an interactive corkboard, interrogating suspects, and discovering hidden clues. The objective of the project was to create a responsive, engaging, and state-driven web experience with a smooth user interface, score-tracking mechanisms, dynamic DOM manipulation, and logical puzzle design.

2. Introduction

Modern web applications rely heavily on dynamic user interactions and real-time state management. The primary goal of this project was to design and implement a lightweight, browser-based detective narrative game centered around a fictional campus event at the Asian University for Women (AUW).

Objectives:

- Design an interactive corkboard UI for structured evidence presentation.
- Implement dynamic logic for searching hidden clues, interrogating suspects, and score deductions.
- Create a progressive difficulty curve for clues (Hard - Hard - Easy/Medium).
- Build a modular codebase separating layout (HTML), styling (CSS), and logic (JavaScript).

3. Methodology / Implementation

The project follows a standard three-tier client-side architecture:

```

📁 detective-game/
├── Final.html      (Structure & DOM elements)
├── Final.css       (Visual styling & layout)
├── Final.js        (Game state & interaction logic)
├── bg-pic.jpg      (Transparent background image)
├── Mittra.jpg.jpeg (Suspect 1 Avatar)
├── Lomi.jpg.jpeg   (Suspect 2 Avatar)
└── Suvodra.jpg.jpeg (Suspect 3 Avatar)
```

System Architecture:

1. **HTML Structure (Final.html):** Divided into a three-column layout: a **Sidebar** (Briefing & Suspects), a **Corkboard Area** (Evidence Cards), and an **Action Pane** (Search Room & Accuse controls).
2. **CSS Styling (Final.css):** Built using CSS Grid and Flexbox. A dedicated overlay layer (.bg-layer) uses CSS opacity and z-index properties to render a transparent background image without lowering the opacity of text or UI controls.
3. **JavaScript Logic (Final.js):** Manages game state variables (Score, HiddenSearchCount), dynamically appends evidence cards to the corkboard DOM, and controls pop-up modal dialogues.

4. Results

The final application successfully delivers an immersive detective experience running smoothly in standard modern browsers.

Key Features Implemented:

- **Interactive Evidence Corkboard:** Clickable cards trigger pop-up modals containing investigation details.
- **Progressive Hidden Search:** Searching the room sequentially reveals two hidden clues:
 1. *Hidden Clue 1:* Left-handed Precision Screwdriver.
 2. *Hidden Clue 2:* Bright Pink Hair Strand trapped in the cable casing.
- **Score System:** Tracks player performance starting at **300 points**. Point deductions occur dynamically based on player choices: inspecting clues or searching room for hidden evidence (**-50 points** per clue) and making false accusations (**-80 points** per wrong attempt).

5. Challenges and Limitations

Challenges Faced:

1. **Background Transparency:** Setting opacity on a parent element caused text and buttons to become faint. This was solved by creating an isolated .bg-layer div behind the UI elements.
2. **Sequential Search Logic:** Preventing duplicate clue creation when searching the room was solved by using a state counter variable (HiddenSearchCount) in JavaScript.

3. **Clue Balance:** Structuring evidence so that solving the case required combining physical evidence (left-handed grip + pink hair strand) rather than naming the culprit directly in early clues.

Limitations:

- **Progress Persistence:** Game progress resets on browser refresh (no Local Storage or database implementation).
- **Single Case Scenario:** The current version features a fixed narrative and single culprit outcome.

6. Conclusion and Discussion

The "Sabotage at AUW Server Room" game successfully demonstrates the integration of HTML, CSS, and JavaScript to create an engaging web-based project. All core goals including DOM manipulation, transparent UI layering, state tracking, and clue progression were successfully met.

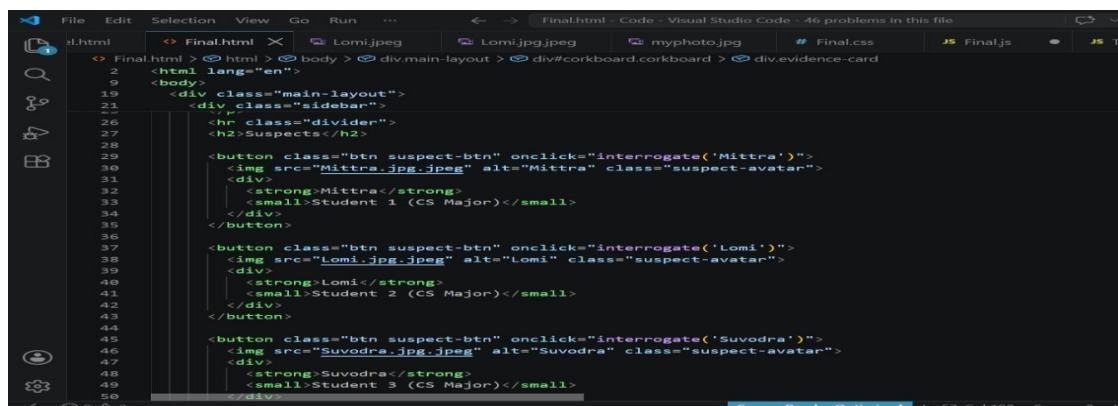
8. References

1. MDN Web Docs. *Manipulating documents in JavaScript*. <https://developer.mozilla.org>
2. W3Schools. *CSS Layout - Flexbox & Grid*. <https://www.w3schools.com>

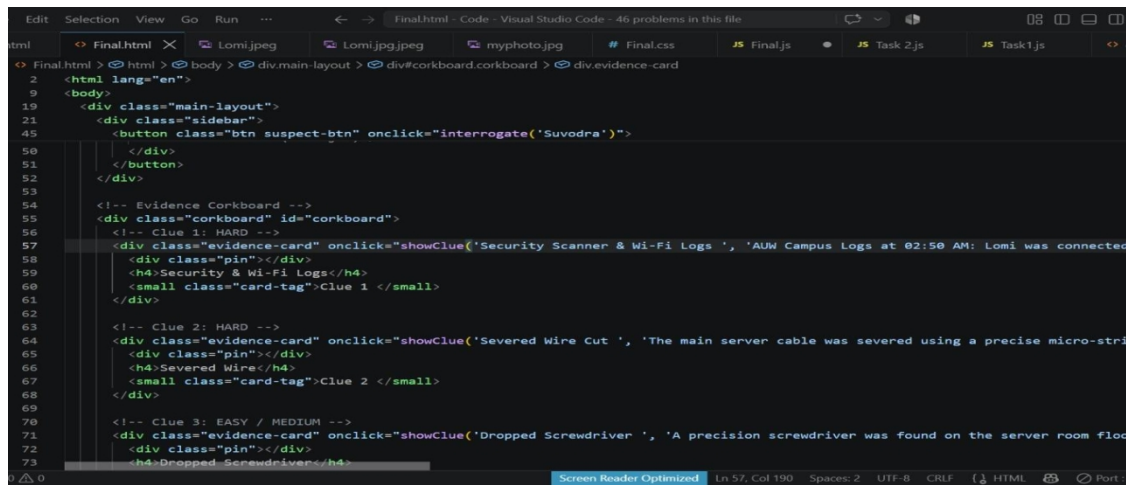
9. Individual Contributions

1. Mittra- HTML Structure & Layout Architecture

- **Responsibilities:** Authored Final.html, designed DOM hierarchy, sidebar layouts, initial corkboard structure, and accusation dropdown controls.
- **Code Implementation & Explanation:** Mittra established the main DOM container and suspect navigation structure:



```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Sabotage at AUW Server Room</title>
6   </head>
7   <body>
8     <div class="main-layout">
9       <div class="sidebar">
10        <hr class="divider">
11        <h2>Suspects</h2>
12        <button class="btn suspect-btn" onclick="interrogate('Mittra')">
13          
14          <strong>Mittra</strong>
15          <small>Student 1 (CS Major)</small>
16        </button>
17        <button class="btn suspect-btn" onclick="interrogate('Lomi')">
18          
19          <strong>Lomi</strong>
20          <small>Student 2 (CS Major)</small>
21        </button>
22        <button class="btn suspect-btn" onclick="interrogate('Suvodra')">
23          
24          <strong>Suvodra</strong>
25          <small>Student 3 (CS Major)</small>
26        </button>
27      </div>
28    </div>
29  </body>
30 </html>
```

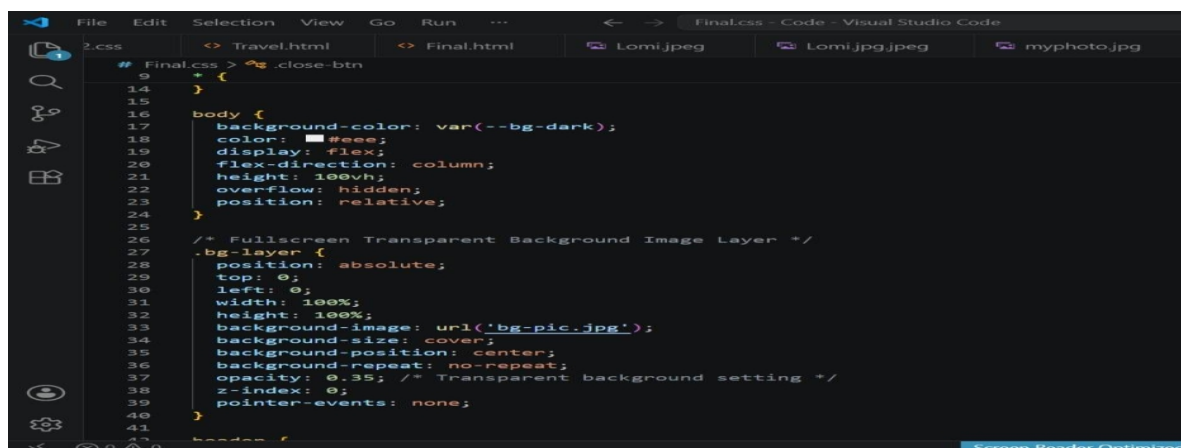


```
Final.html > body > div.main-layout > div.corkboard.corkboard > div.evidence-card
2 <html lang="en">
9 <body>
19 <div class="main-layout">
21 <div class="sidebar">
45 <button class="btn suspect-btn" onclick="interrogate('Suvodra')">
50 </div>
51 </button>
52 </div>
53
54 <!-- Evidence Corkboard -->
55 <div class="corkboard" id="corkboard">
56 <!-- Clue 1: HARD -->
57 <div class="evidence-card" onclick="showClue('Security Scanner & Wi-Fi Logs ', 'AUW Campus Logs at 02:50 AM: Lomi was connected
58 <div class="pin"></div>
59 <h4>Security & Wi-Fi Logs</h4>
60 <small class="card-tag">Clue 1 </small>
61 </div>
62
63 <!-- Clue 2: HARD -->
64 <div class="evidence-card" onclick="showClue('Severed Wire Cut ', 'The main server cable was severed using a precise micro-str
65 <div class="pin"></div>
66 <h4>Severed Wire</h4>
67 <small class="card-tag">Clue 2 </small>
68 </div>
69
70 <!-- Clue 3: EASY / MEDIUM -->
71 <div class="evidence-card" onclick="showClue('Dropped Screwdriver ', 'A precision screwdriver was found on the server room flo
72 <div class="pin"></div>
73 <h4>Dropped Screwdriver</h4>
```

Explanation: Mittra established clean semantic HTML IDs (corkboard, score, accuseSelect) and attribute hooks (onclick) that allowed seamless DOM manipulation by Suvodra's JavaScript script while applying Lawmi's CSS style rules

2. Lawmi- CSS Styling, Layout & Visual Design

- **Responsibilities:** Authored `Final.css`, created visual themes, card pin effects, modal pop-ups, score badges, and transparent background layers.
- **Code Implementation & Explanation:** Lawmi designed the background opacity layer to prevent text transparency issues:

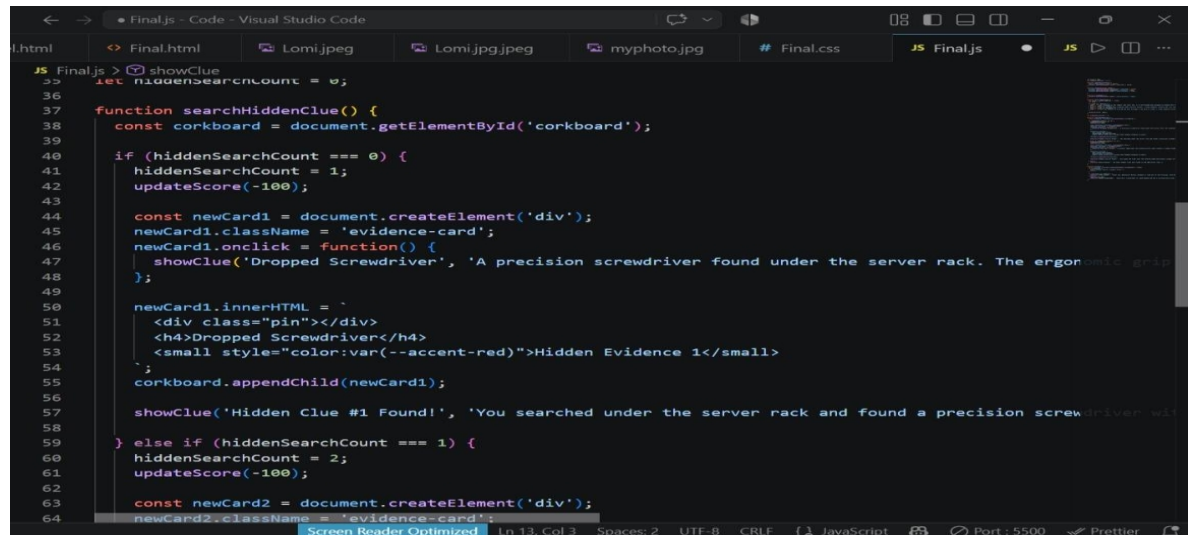


```
Final.css > .close-btn
9 + {
14 }
15
16 body {
17 background-color: var(--bg-dark);
18 color: #eee;
19 display: flex;
20 flex-direction: column;
21 height: 100vh;
22 overflow: hidden;
23 position: relative;
24 }
25
26 /* Fullscreen Transparent Background Image Layer */
27 .bg-layer {
28 position: absolute;
29 top: 0;
30 left: 0;
31 width: 100%;
32 height: 100%;
33 background-image: url('bg-pic.jpg');
34 background-size: cover;
35 background-position: center;
36 background-repeat: no-repeat;
37 opacity: 0.35; /* Transparent background setting */
38 z-index: 0;
39 pointer-events: none;
40 }
41
```

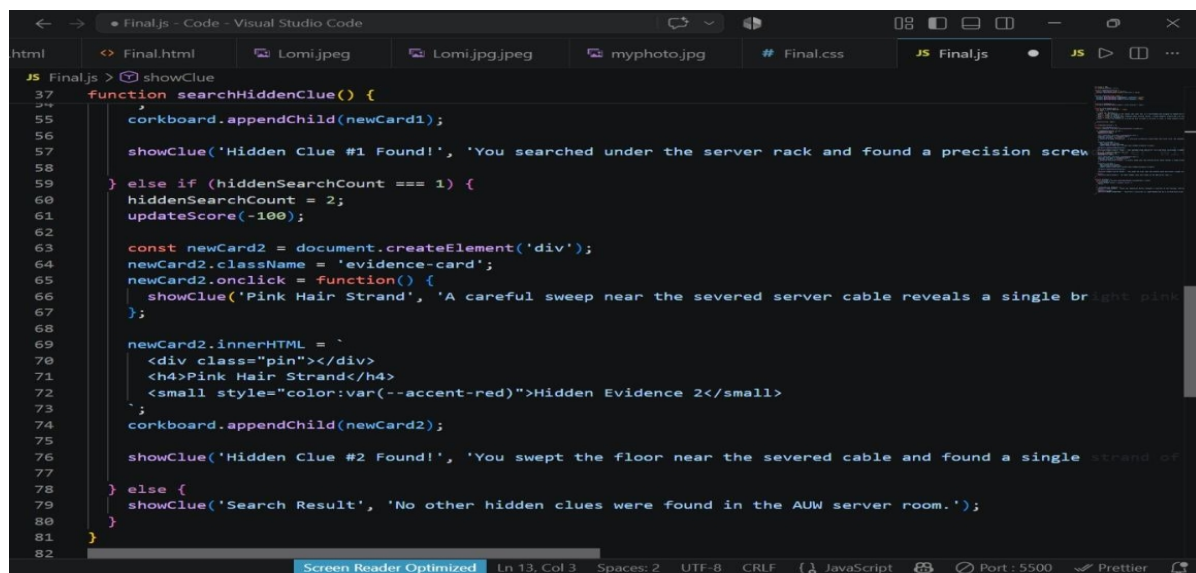
Explanation: Lawmi used an independent absolute-positioned `.bg-layer` div placed behind UI elements (z-index: 0) with `pointer-events: none` and `opacity: 0.35`. This rendered `bg-pic.jpg` transparently across the screen without making text or UI buttons transparent or unclickable.

3. Suvodra - JavaScript Development & Logic Implementation

- **Responsibilities:** Authored `Final.js`, implemented state management, score tracking, modal dialog logic, and the sequential hidden search function.
- **Code Implementation & Explanation:** Suvodra developed the multi-step search function allowing players to discover two distinct hidden clues sequentially:



```
Final.js > showClue
35 let hiddenSearchCount = 0;
36
37 function searchHiddenClue() {
38   const corkboard = document.getElementById('corkboard');
39
40   if (hiddenSearchCount === 0) {
41     hiddenSearchCount = 1;
42     updateScore(-100);
43
44     const newCard1 = document.createElement('div');
45     newCard1.className = 'evidence-card';
46     newCard1.onclick = function() {
47       showClue('Dropped Screwdriver', 'A precision screwdriver found under the server rack. The ergonomic grip
48     );
49
50     newCard1.innerHTML = `
51       <div class="pin"></div>
52       <h4>Dropped Screwdriver</h4>
53       <small style="color:var(--accent-red)">Hidden Evidence 1</small>
54     `;
55     corkboard.appendChild(newCard1);
56
57     showClue('Hidden Clue #1 Found!', 'You searched under the server rack and found a precision screwdriver with
58   } else if (hiddenSearchCount === 1) {
59     hiddenSearchCount = 2;
60     updateScore(-100);
61
62     const newCard2 = document.createElement('div');
63     newCard2.className = 'evidence-card';
```



```
Final.js > showClue
37 function searchHiddenClue() {
55   corkboard.appendChild(newCard1);
56
57   showClue('Hidden Clue #1 Found!', 'You searched under the server rack and found a precision screw
58
59 } else if (hiddenSearchCount === 1) {
60   hiddenSearchCount = 2;
61   updateScore(-100);
62
63   const newCard2 = document.createElement('div');
64   newCard2.className = 'evidence-card';
65   newCard2.onclick = function() {
66     showClue('Pink Hair Strand', 'A careful sweep near the severed server cable reveals a single bright pink
67   );
68
69   newCard2.innerHTML = `
70     <div class="pin"></div>
71     <h4>Pink Hair Strand</h4>
72     <small style="color:var(--accent-red)">Hidden Evidence 2</small>
73   `;
74   corkboard.appendChild(newCard2);
75
76   showClue('Hidden Clue #2 Found!', 'You swept the floor near the severed cable and found a single strand of
77
78 } else {
79   showClue('Search Result', 'No other hidden clues were found in the AUW server room.');
```

Explanation: This function tracks the variable hiddenSearchCount. On the first click, it creates and appends "Hidden Evidence 1" (Screwdriver) to the corkboard DOM. On the second click, it appends "Hidden Evidence 2" (Pink Hair Strand). Subsequent clicks notify the player that no more clues exist, preventing infinite point deductions.

